Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Kuyruk türleri: girişi kısıtlı (input restricted), çıkışı kısıtlı (output restricted), çift uçlu (double ended) ve öncelikli (priority) kuyruk.
- Doğrusal olmayan (non-linear) veri yapılarına giriş: ağaçlar (tree).
- Ağaç terminolojisi ve ağaçların kullanım alanları.
- Ağaçta gezinme (tree traversal): inorder, preorder, postorder ve seviye seviye gezinme.
- Özyinelemeli gezinme fonksiyonları.
- İkili ağaç türleri: full, perfect, complete, balanced, degenerate ve skewed.
- İkili arama ağacı (binary search tree, BST): arama, ekleme ve silme.
- Kendini dengeleyen ağaçlar: AVL, kırmızı-siyah (red-black), B ve B+ ağaçları.

Hocanın Özellikle Vurguladığı Kısımlar

- Her ağaç bir graftır, ama her graf ağaç değildir.
- Ağacın bir tarafa yaslı (skewed) olması istenmez. Bu durumda işlemler $O(\log n)$ yerine $O(n)$ olur; diziden farkı kalmaz.
- Mümkün olduğunca dengeli ağaçlar kullanılır ya da ağaç dengeli hale getirilir.
- Ağaçta gezinmeyi özyinelemeli (recursive) yazmak çok daha kolaydır. Ancak çok derin ağaçta stack overflow riski vardır.
- İkili arama ağacı (BST) ile ikili arama (binary search) algoritması farklı kavramlardır.
- Dizi indeksinin 0'dan mı 1'den mi başladığı çocuk indeks formülünü değiştirir. Varsayımınızı her zaman belirtin.
- Aynı indeksi hem girdi dizisi hem ağaç dizisi için kullanmak ciddi bir hatadır ("algoritma cinayeti").
- BST'de silmenin en zor durumu, iki çocuklu düğümün silinmesidir.

Kısa Tekrar Notları

- Kuyruk türleri:
  - Girişi kısıtlı kuyruk: ekleme tek uçtan, çıkarma iki uçtan.
  - Çıkışı kısıtlı kuyruk: ekleme iki uçtan, çıkarma tek uçtan.
  - Çift uçlu kuyruk (deque): iki uçtan da ekleme ve çıkarma.
  - Öncelikli kuyruk: en yüksek (veya en düşük) öncelikli eleman en tepede; heap ile gerçeklenir.
- Ağaç terimleri:
  - Kök (root), kenar (edge), düğüm (vertex/node).
  - Seviye (level): kök 0. seviyededir.
  - Yol (path), ebeveyn (parent), çocuk (child), ata (ancestor), kardeş (sibling).
  - Alt ağaç (subtree), yaprak (leaf), orman (forest).
  - Ziyaret (visiting) ve gezinme (traversing).
- Gezinme sıraları:
  - Inorder: sol, düğüm, sağ.
  - Preorder: düğüm, sol, sağ.
  - Postorder: sol, sağ, düğüm.
- İkili ağaçta her düğümün en fazla 2, üçlü (ternary) ağaçta en fazla 3, n'li (n-ary) ağaçta en fazla n çocuğu olur.

- 0'dan başlayan dizide `i` düğümünün sol çocuğu `2i + 1`, sağ çocuğu `2i + 2`'dir.
- BST kuralı: sol alt ağaçtaki her değer düğümden küçük, sağ alt ağaçtaki her değer düğümden büyüktür.

Detaylı Açıklamalar

Kuyruk Türleri

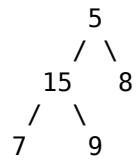
- Girişi kısıtlı kuyrukta normal kuyruğa ek olarak son eklenen eleman da çıkarılabilir.
  - Örnek: ağ tamponunda (buffer) çok bekleyen ya da kuyruğu tıkayan paketleri sondan düşürmek.
- Çıkışı kısıtlı kuyrukta başa da eleman eklenebilir.
  - Örnek: yüksek öncelikli bir eleman geldiğinde onu kuyruğun önüne koymak.
- Çift uçlu kuyruk, bu iki yapının birleşimidir.
- Öncelikli kuyruk heap ağacı ile gerçeklenir; tepede her zaman ilk hizmet alacak eleman bulunur. Ayrıntılar heap konusunda anlatılacak.

Ağaçların Kullanım Alanları

- Dosya sistemi: en üstte kök dizin (`/`), altında `home`, `etc` gibi klasörler.
- Huffman kodlaması ile sıkıştırma.
- Veritabanı indeksleme.
- Derleyicilerde sözdizimi ağacı (syntax tree).
- Öncelikli kuyruk için heap.
- Yapay zekada karar ağaçları (decision tree) ve rastgele orman (random forest).
- Coğrafi bilgi sistemlerinde k-d ağaçları: kıta, ülke, şehir gibi hiyerarşiyle aramayı $O(n)$'den $O(\log n)$'e indirmek.

Ağaçta Gezinme

Örnek ağaç:



Gezinme	Sonuç
Inorder	7, 15, 9, 5, 8
Preorder	5, 15, 7, 9, 8
Postorder	7, 9, 15, 8, 5
Seviye seviye, yukarıdan aşağı	5, 15, 8, 7, 9
Seviye seviye, aşağıdan yukarı	7, 9, 15, 8, 5

Düğüm yapısı ve özyinelemeli gezinme:

```c
struct node {
    int data;
    struct node *left;
    struct node *right;
};

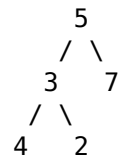
void inorder(struct node *root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}
```

```c
void preorder(struct node *root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

void postorder(struct node *root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}
```

- Durma koşulu (base case) `root == NULL` olmasıdır.
- Üç fonksiyonun tek farkı `printf` satırının yeridir.

İkinci örnek ağaç:



Gezinme	Sonuç
Inorder	4, 3, 2, 5, 7
Preorder	5, 3, 4, 2, 7
Postorder	4, 2, 3, 7, 5

İkili Ağaç Türleri

- **Full binary tree:** Her düğümün 0 ya da 2 çocuğu vardır.
- **Perfect binary tree:** Tüm iç düğümlerin 2 çocuğu vardır ve tüm yapraklar aynı seviyedir.
- **Complete binary tree:** Son seviye dışındaki tüm seviyeler doludur. Son seviyedeki düğümler sola yaslı ve arada boşluk olmadan dizilir.
- **Balanced binary tree:** Her düğümde sol ve sağ alt ağaçların yükseklik farkı en fazla 1'dir.
- **Degenerate (pathological) tree:** Her iç düğümün tek çocuğu vardır.
- **Skewed binary tree:** Degenerate ağacın özel hali; tüm düğümler tek tarafa (sola ya da sağa) yaslıdır.

Derste incelenen örneklerin özeti:

Durum	Full	Complete
Bir düğümün tek çocuğu var, son seviyede boşluk var	Hayır	Hayır
Her düğümün 0 ya da 2 çocuğu var, son seviye sola yaslı değil	Evet	Hayır
Bir düğümün tek (sol) çocuğu var, son seviye sola yaslı	Hayır	Evet
Her düğümün 0 ya da 2 çocuğu var, son seviye sola yaslı	Evet	Evet

- Yükseklik farkı 1 olan ağaç dengeli kabul edilir. Fark 2 ve üzeri olursa dengesizdir.

- Dengesiz ama bir tarafa yaslı olmayan ağaç kabul edilebilir olabilir. Ağacı ne zaman yeniden dengelemek gerektiği ayrı bir karardır.

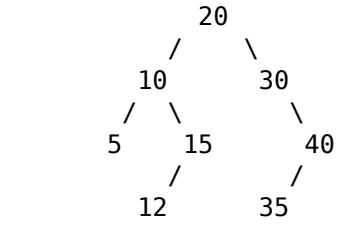
BST'de Arama

- Kökten başlanır. Aranan değer düğüme eşitse bulunmuştur.
- Küçükse sola, büyükse sağa gidilir.
- Örnek: 26 aranıyor. Yol: 15'ten sağa, 21'den sağa, 27'den sola, 25'ten sağa.
- Durma koşulu veri yapısına bağlıdır:
  - Dizi ile saklanıyorsa boş gözler -1 ile işaretlenir; -1'e gelinirse eleman yoktur.
  - Bağlı yapı ile saklanıyorsa `NULL` görülünce durulur.

```
int i = 0;
while (tree[i] != -1 && tree[i] != x) {
    if (x < tree[i]) i = 2 * i + 1;   /* sol çocuk */
    else             i = 2 * i + 2;   /* sağ çocuk */
}
if (tree[i] == -1) printf("not found\n");
else               printf("%d. gozde bulundu\n", i);
```

BST'ye Ekleme (Diziden Ağaç Kurma)

Girdi: `20, 10, 30, 5, 15, 40, 12, 35`.



Derste birlikte düzeltilen akışın özü:

```
for (int k = 0; k < SIZE; k++) tree[k] = -1;

for (int j = 0; j < n; j++) {
    int i = 0;                          /* her eleman için kökten başla */
    while (tree[i] != -1) {
        if (a[j] > tree[i]) i = 2 * i + 2;
        else                i = 2 * i + 1;
    }
    tree[i] = a[j];
}
```

Tahtadaki ilk taslakta bulunan hatalar:

- Girdi dizisi ile ağaç dizisi aynı indeksle (`i`) geziliyordu. Girdi için ayrı bir indeks (`j`) gerekir.
- Her yeni eleman için ağaç indeksi kökten (0) yeniden başlatılmıyordu.
- Sol ve sağ çocuk formülleri (`2i + 1` ve `2i + 2`) ters yazılmıştı.
- Dizi C'de 0'dan başladığı için döngü sınırları 0 ile $n-1$ olmalıydı.

BST'den Silme

Örnek ağaç: kök 5; sol 3 (çocukları 2 ve 4); sağ 12 (çocukları 8 ve 17); 17'nin çocukları 16 ve 18.

- **Durum 1, yaprak silme:** Yalnızca düğüm kaldırılır. Örnek: 2'yi silmek için gözüne -1 yazılır.
- **Durum 2, tek çocuklu düğüm:** Çocuk, silinen düğümün yerine geçer. Örnek: 18 yoksa 17 silinince 16 yukarı çıkar.

- **Durum 3, iki çocuklu düğüm:** Sağ alt ağacın en küçük elemanı (inorder ardıl, inorder successor) bulunur ve silinen düğümün yerine konur. Bu eleman, sağ alt ağaçta hep sola gidilerek bulunur.
  - Örnek: 20 silinirken yerine sağ alt ağacının en küçüğü olan 25 konur.
  - Sol alt ağaçtan bir değer (örneğin 16) konursa BST kuralı bozulabilir.
- **Durum 4, eleman ağaçta yok:** Arama -1'e (ya da `NULL`'a) ulaşır; eleman bulunamadı denir.
- Silme işlemi ağacın dengeli kalacağını garanti etmez. Gerekirse dengeleme algoritması çalıştırılır.

Kendini Dengeleyen Ağaçlar

- **AVL ağacı:** Kendini dengeleyen BST.
- **Kırmızı-siyah ağaç (red-black tree):** Kendini dengeleyen BST. Her düğümde rengini tutan ek bir bit vardır; denge bu renklerle kontrol edilir.
- **B ağacı (B-tree):** Erişim, ekleme ve silmeyi verimli yapan, kendini dengeleyen çok çocuklu ağaç.
- **B+ ağacı:** Dosya sistemlerinde ve veritabanlarında kullanılır.
- Bu ağaçların ayrıntıları dersin konusu değil; hoca göz atmayı önerdi.
- Gelecek hafta: heap veri yapısı, öncelikli kuyruk ve vakit kalırsa heap sort.

Ders sonrası notlar

- **Sol alt ağacın en büyüğü:** Hoca, iki çocuklu düğüm silinirken sol alt ağacın en büyük elemanının (inorder öncül, inorder predecessor) her durumda işe yaramayacağını söyledi ve bunu gelecek hafta kontrol edeceğini belirtti. Doğrusu: öncül de her durumda doğru sonuç verir. Öncül, sol alt ağaçtaki tüm değerlerden büyük ve sağ alt ağaçtaki tüm değerlerden küçüktür; BST kuralı korunur. Ardıl ya da öncül seçimi bir tercih meselesidir.
- **Ardılın çocuğu:** Ardıl her zaman yaprak değildir; sağ çocuğu olabilir. Ardıl yerine taşındıktan sonra bu sağ çocuk, ardılın eski yerine bağlanmalıdır.
- **Dizi gösteriminde taşıma:** Dizi ile saklanan ağaçta tek çocuklu düğüm silinirken yalnız bir göz değil, çocuğun bütün alt ağacı bir seviye yukarı taşınmalıdır. Bağlı yapıda yalnız bir işaretçi değişir.
- **Dengeli ağaç tanımı:** Hoca tanımda alt ağaçların "boyutundan" söz etti. Ölçüt, düğüm sayısı değil alt ağaçların yüksekliğidir ve koşul her düğüm için sağlanmalıdır.
- **Dizi sınırı kontrolü:** Arama ve ekleme döngülerinde (derste çizilen akışta da) `i`'nin dizi boyutunu aşıp aşmadığı kontrol edilmez. Ağaç derinleşirse `tree[i]` dizinin dışına taşar. Döngü koşuluna `i < SIZE` kontrolü eklenmelidir. Derslerdeki 20, 10, 30, 5, 15, 40, 12, 35 girdisinde en büyük indis 13'tür.
- **Dizi boyutu:** Hoca n elemanlık bir BST için en kötü durumda 2^n göz gerektiğini söyledi. Tamamen tek tarafa yaslı ağaçta en derin düğümün indeksi $2^n - 2$ olur; yani $2^n - 1$ göz yeter. Sonuç yine üstel büyüklüktedir; dizi gösterimi dengesiz ağaçlar için israflıdır.
- **Özyineleme ve verimlilik:** Derste özyinelemeli gezinmenin "daha verimli" olduğu söylendi. Özyineleme kodu kısaltır ve kolaylaştırır; çalışma süresi yineli (iterative) çözümle aynı mertebededir ($O(n)$), ek olarak yığın alanı kullanır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.